OPERATING SYSTEMS

ORANGE LEVEL PROBLEM

Sakaray Vishal
PES1UG24AM241
AIML 4E

SCREENSHOT 1A:

```
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B:

```
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$
```

2A:

```
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$ make test_tree
./test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$
```

2B:

```
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$ xxd .pes/objects/0b/d69098bd9b9cc5934a610ab65da429b52
5361147faa7b5b922919e9a23143d | head -20
00000000: 626c 6f62 2031 3200 6865 6c6c 6f20 776f  blob 12.hello wo
00000010: 726c 640a                                rld.
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$
```

3A:

```
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index    # Human-readable text format
make: 'pes' is up to date.
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file.txt
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h

100755 0bd69098bd9b9cc5934a610ab65da429b5253611147faa7b5b922919e9a23143d 1776352618 12 file.txt
100755 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776352701 6 file1.txt
```

3B:

```
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$ cat .pes/index
100755 0bd69098bd9b9cc5934a610ab65da429b5253611147faa7b5b922919e9a23143d 1776352618 12 file.txt
100755 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776352701 6 file1.txt
100755 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776352701 6 file2.txt
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$
```

4A:

```
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 8ab4d5e63d4d... Initial commit
Committed: f6ae7987bfd1... Add world
Committed: e68bb400e475... Add farewell
commit e68bb400e475e1e8355250bb43f88033891cb09c16b2c5c579f62bb6a50bc585
Author: Sakaray Vishal <PES1UG24AM241>
Date:   1776352998

    Add farewellver auth!

commit f6ae7987bfd1ae85d5acabc857dfc2529b48dee8c1ba9546c3616d9f8f175e3c
Author: Sakaray Vishal <PES1UG24AM241>
Date:   1776352998

    Add world

commit 8ab4d5e63d4d79425f98a4390a9791c1b5463e6528f4dd6fca8f35d9733545e6
Author: Sakaray Vishal <PES1UG24AM241>
Date:   1776352998

    Initial commit

commit 21d6b929fa588569c874a63a5e4929c5a7b1b6b0831bf2dbe62bd2b0d3ee4667
Author: Sakaray Vishal <PES1UG24AM241>
Date:   1776352968

    Add farewellit

commit 23f66a99a39535d9c1f8c6f58234124dd4fe7fe762367197ac794052d343c1a8
Author: Sakaray Vishal <PES1UG24AM241>
Date:   1776352968

    Add world

Add world!

commit d96be44c0db73228ccbd84c3169694bbafb5e590e445012a439d59725aac991d
Author: Sakaray Vishal <PES1UG24AM241>
Date:   1776352968

    Initial commit

v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$
```

4B:

```
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/21/d6b929fa588569c874a63a5e4929c5a7b1b6b0831bf2dbe62bd2b0d3ee4667
.pes/objects/23/f66a99a39535d9c1f8c6f58234124dd4fe7fe762367197ac794052d343c1a8
.pes/objects/55/8c1061e21bccbeb23c527d8dcef5c8646cabfb6091aae3ae7ee3320c9ff7b7
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7d/27cda7980acf0e76401ab6218f9c7364891b8524166b21775841078d9b86a0
.pes/objects/8a/b4d5e63d4d79425f98a4390a9791c1b5463e6528f4dd6fca8f35d9733545e6
.pes/objects/cc/e49eb2943f54e8135f291862bdf218064409ba05a7ee7d5bec00ddc22addfb
.pes/objects/d9/6be44c0db73228ccbd84c3169694bbafb5e590e445012a439d59725aac991d
.pes/objects/dd/82340b5e403e2dc18bfca050e6699935849710b2b5d28ab8bbffbb72d32627
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/e6/8bb400e475e1e8355250bb43f88033891cb09c16b2c5c579f62bb6a50bc585
.pes/objects/f6/ae7987bfd1ae85d5acabc857dfc2529b48dee8c1ba9546c3616d9f8f175e3c
.pes/refs/heads/main
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$
```

4C:

```
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$ cat .pes/refs/heads/main && cat .pes/HEAD
e68bb400e475e1e8355250bb43f88033891cb09c16b2c5c579f62bb6a50bc585
ref: refs/heads/main
```

FINAL INTEGRATION:

```
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$ make test-integratio
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 7870f2800188... Initial commit

Log after first commit:
commit 7870f2800188e27ec093a41a78f9b4e117ee7763fdc144a5013de83c58879533
Author: Sakaray Vishal <PES1UG24AM241>
Date:   1776353204

    Initial commit


--- Second Commit ---
Committed: bfc9cc01af8d... Update file.txt

--- Third Commit ---
Committed: f74488301874... Add farewell

--- Full History ---
commit f74488301874ba08fe45d2f49ec41026cb7eaecf3b832243e3c94ca03ed611ea
Author: Sakaray Vishal <PES1UG24AM241>
Date:   1776353204

    Add farewellver auth!

commit bfc9cc01af8d83e11f805e3376f1b5afffc311261cae154f44939df38b4ffb7e
Author: Sakaray Vishal <PES1UG24AM241>
Date:   1776353204

    Update file.txt auth!

commit 7870f2800188e27ec093a41a78f9b4e117ee7763fdc144a5013de83c58879533
Author: Sakaray Vishal <PES1UG24AM241>
Date:   1776353204

    Initial committial commitES1A


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
f74488301874ba08fe45d2f49ec41026cb7eaecf3b832243e3c94ca03ed611ea

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/78/70f2800188e27ec093a41a78f9b4e117ee7763fdc144a5013de83c58879533
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/bf/c9cc01af8d83e11f805e3376f1b5afffc311261cae154f44939df38b4ffb7e
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f7/4488301874ba08fe45d2f49ec41026cb7eaecf3b832243e3c94ca03ed611ea

=== All integration tests completed ===
v1@Swift:/mnt/c/Users/sakar/Desktop/os/PES1UG24AM241-pes-vcs-main$
```

Questions answers –

Q5.1: How would you implement pes checkout <branch>?

A)To implement pes checkout <branch>, three things must change inside .pes/:

1. Read the commit hash from .pes/refs/heads/<branch>
2. Update .pes/HEAD to contain ref: refs/heads/<branch>
3. Rewrite the index file to reflect the target commit's tree

For the working directory, the system must walk the target commit's root tree, compare each file against the current working directory, then create, overwrite, or delete files as needed so the working directory matches the target snapshot.

The operation is complex because it must be atomic across many files — a crash mid-checkout can leave a mix of old and new files, the index can go out of sync with the working directory, untracked local files must not be destroyed, and conflicts with uncommitted changes must be detected before touching anything

Q5.2 — Detecting a dirty working directory

A) A file is "dirty" if it differs from both the index and the target branch's tree. To detect this, iterate through each entry in the current index. For each path, compare three versions:

1. **Index hash** — from .pes/index
2. **Working directory hash** — recomputed by reading the actual file and hashing it (or skip the hash and compare mtime + size for a fast check)
3. **Target tree hash** — from the tree of the branch being checked out to

If working directory hash ≠ index hash (file modified locally but not staged) AND index hash ≠ target tree hash (file also differs between

branches), checkout must refuse because applying the target would silently discard the local changes.

Q5.3 — Detached HEAD

A) Detached HEAD means .pes/HEAD contains a raw commit hash instead of ref: refs/heads/<branch>. Commits made in this state still write normally to the object store and head_update still overwrites HEAD with the new commit hash — but since no branch ref points to these commits, they are "floating." If the user switches to another branch, those commits become unreachable (no branch sees them) and will be deleted on the next garbage collection.

Recovery: before switching branches, the user must note the commit hash (via pes log or the current value of HEAD) and create a new branch pointing to it — e.g., pes branch rescue <hash> — which writes the hash into .pes/refs/heads/rescue, making those commits reachable again.

Q6.1 — Garbage collection algorithm

A) Algorithm: **mark and sweep**.

1. **Mark phase:** start from all branch refs (.pes/refs/heads/*) and HEAD. For each ref, read its commit, mark the commit hash, then recursively mark its tree, all subtrees inside the tree, all blobs referenced by those trees, and finally follow the commit's parent pointer. Continue until every reachable object is marked.
2. **Sweep phase:** walk .pes/objects/ on disk. For every object file, check if its hash is in the "reachable" set. If not, delete it.

The ideal data structure for the reachable set is a **hash set (open-addressed hash table keyed by the 32-byte ObjectID)**, giving O(1) lookup.

For a repo with 100,000 commits and 50 branches: each commit references ~1 tree and on average a few subtrees/blobs. Assuming branches share most history and an average of ~5 objects referenced per commit, the walk visits roughly 100,000 commits + 500,000 trees/blobs ≈

600,000 objects. The 50 branches add little extra work because they share ancestry — the walk simply starts from 50 tips instead of 1.

Q6.2 — GC vs. concurrent commit race condition

A) Race condition: a commit operation has three steps — write blob objects, write a tree, write a commit. There is a window between writing a blob (say hello.txt) and writing the tree that references it. If GC runs during that window, it sees the blob on disk but finds no tree or commit referring to it, marks it as unreachable, and deletes it. The commit then writes a tree pointing to a blob hash that no longer exists, producing a corrupt repository.

Real Git avoids this using three mechanisms:

1. **Reachability grace period** — objects are only considered for deletion if their mtime is older than a cutoff (default two weeks), so freshly written blobs are never collected.
2. **Locks** — GC acquires a repository-wide lock (gc.pid), and commit operations check for it before writing.
3. **Reflog** — every ref update is logged, so even objects not yet referenced by a commit are kept reachable via the reflog entry until the reflog expires.